

#07 Stack, Queue, Deque

김성수

4thrunyul@gmail.com

0. 목차



1. Stack

2. Queue

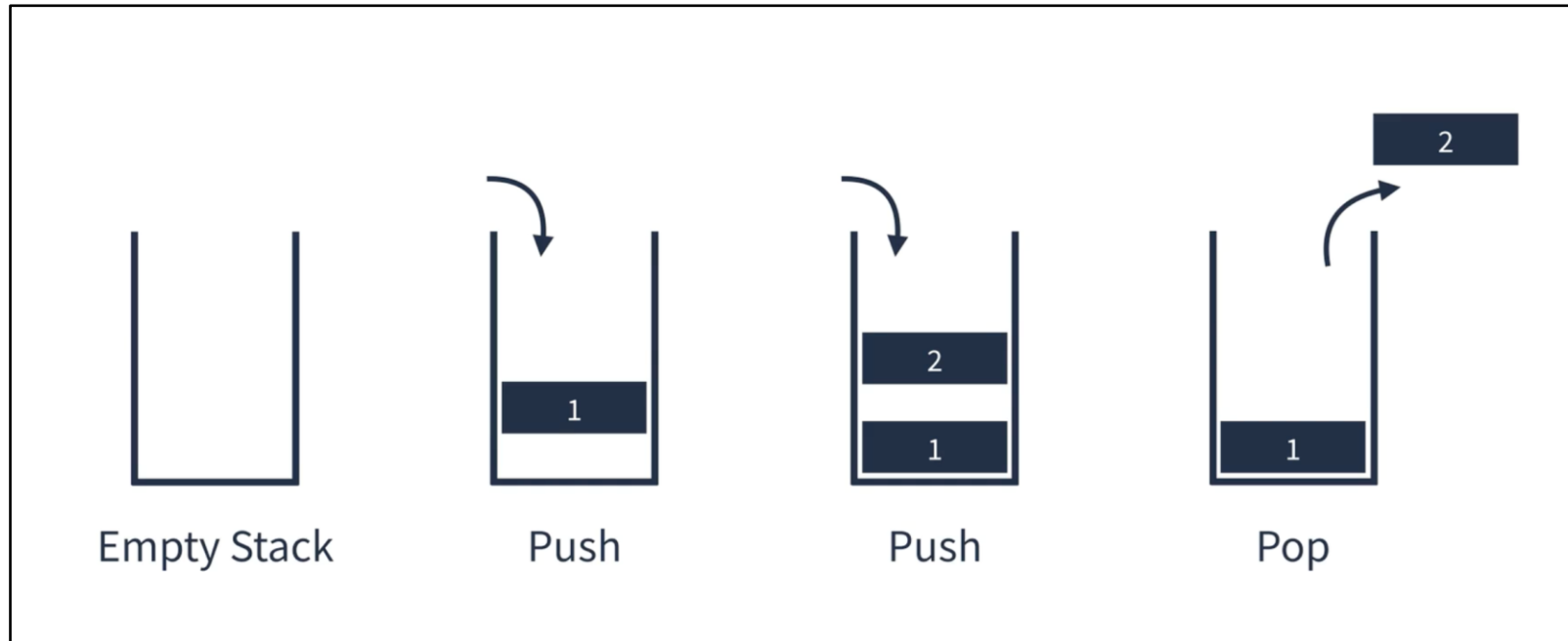
3. Deque

1. Stack

1. Stack

Stack(스택)

- 데이터를 한쪽 끝에서만 넣고 뺄 수 있는 후입선출(LIFO, Last-In First-Out) 방식의 자료구조
- 컴퓨터는 stack을 통해 연산 순서를 정렬함(중위 표기식 -> 후위 표기식)



1. Stack

```

1  #include <iostream>
2  #include <stack>
3
4  using namespace std;
5
6  int main() {
7      stack<int> stk;
8
9      stk.push(1);
10     stk.push(2);
11     stk.pop();
12
13     return 0;
14 }

```

stack<자료형> 변수명 | <stack>

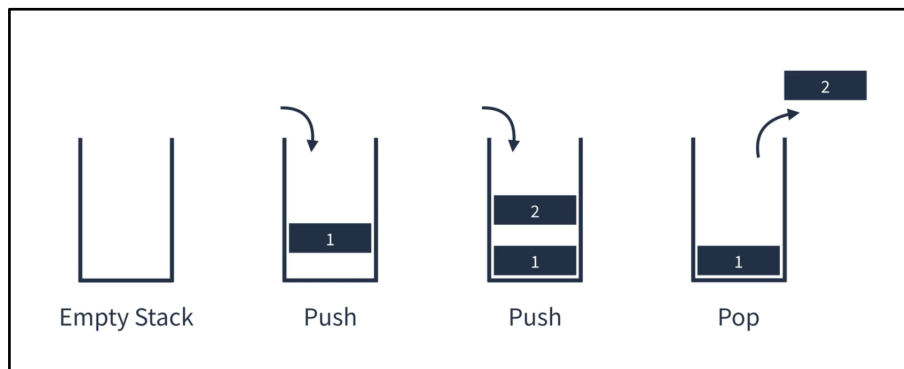
- 스택을 선언하기 위해서는 <stack> 라이브러리 필요
- 자료형에는 거의 모든 자료형을 사용할 수도 있으며, 구조체를 선언하여 자료형으로 넣을 수도 있음
- stack<자료형> 변수명({요소1, 요소2, 요소3}) 처럼 초기 선언도 가능하며, 이 경우 요소1부터 순서대로 push함.

stk.push(요소) | <stack> | O(1)

- 스택의 가장 윗부분에 새로운 데이터를 추가하여 저장하는 함수

stk.pop() | <stack> | O(1)

- 스택의 가장 윗부분에 위치한 데이터를 제거하는 작업
- 비어 있는 스택에 pop을 사용하면 프로그램이 정지되며 아무것도 출력되지 않음



1. Stack

```
1  #include <iostream>
2  #include <stack>
3
4  using namespace std;
5
6  int main() {
7      stack<int> stk;
8
9      stk.push(1);
10     stk.push(2);
11
12     cout << stk.top() << "\n";
13     cout << stk.size() << "\n";
14     cout << stk.empty();
15
16     return 0;
17 }
```

Input:

Copy

Expected Output:

Copy

2
2
0

`stk.top()` | `<stack>` | $O(1)$

- 스택의 가장 윗부분에 위치한 데이터를 조회하는 함수
- 비어 있는 스택에 top을 사용하면 프로그램이 정지되며 아무것도 출력되지 않음

`stk.size()` | `<stack>` | $O(1)$

- 스택에 현재 저장되어 있는 데이터의 총 개수를 확인하는 함수
- 내부 컨테이너에 요소의 수를 저장하고 있어서 $O(1)$ 의 시간 복잡도를 가짐

`stk.empty()` | `<stack>` | $O(1)$

- 스택에 저장된 데이터가 없는지 여부를 확인하는 함수
- bool형으로 반환하며, 스택이 비어 있으면 true, 요소가 하나도 존재한다면 false를 반환

스택의 초기화 | $O(n)$

- 스택에는 `stk.clear()` 함수가 존재하지 않음
- 스택을 초기화하기 위해서는 재선언 하거나 `stk = {}`으로 초기화할 수 있음
- 단, `stk = {1, 2, 3}` 같은 방법은 통하지 않고, `stk = {}`만 예외적으로 가능함.

1. Stack

- [28278 스택 2](#)
- [코드](#)

```
1  #include <iostream>
2  #include <stack>
3
4  using namespace std;
5
6  int main() {
7      ios_base::sync_with_stdio(false);
8      cin.tie(NULL);
9
10     int n, com, num;
11     stack<int> stk;
12
13     cin >> n;
```

```
15     for(int i = 0; i < n; i++) {
16         cin >> com;
17
18         if(com == 1) {
19             cin >> num;
20
21             stk.push(num);
22         }
23         else if(com == 3) {
24             cout << stk.size() << "\n";
25         }
26         else if(!stk.empty()) {
27             if(com == 2) {
28                 cout << stk.top() << "\n";
29
30                 stk.pop();
31             }
32             else if(com == 5) cout << stk.top() << "\n";
33             else if(com == 4) cout << stk.empty() << "\n";
34         }
35         else {
36             if(com == 4) cout << stk.empty() << "\n";
37             else cout << "-1\n";
38         }
39     }
40
41     return 0;
42 }
```

1. Stack: 실습(BOJ)



실습

- [10773 제로](#)
 - [코드](#)
- [9012 괄호](#)
 - [코드](#)
 - Stack을 사용하지 않고 푸는 방법도 고민해보기

1. Stack: 실습(정올)



실습

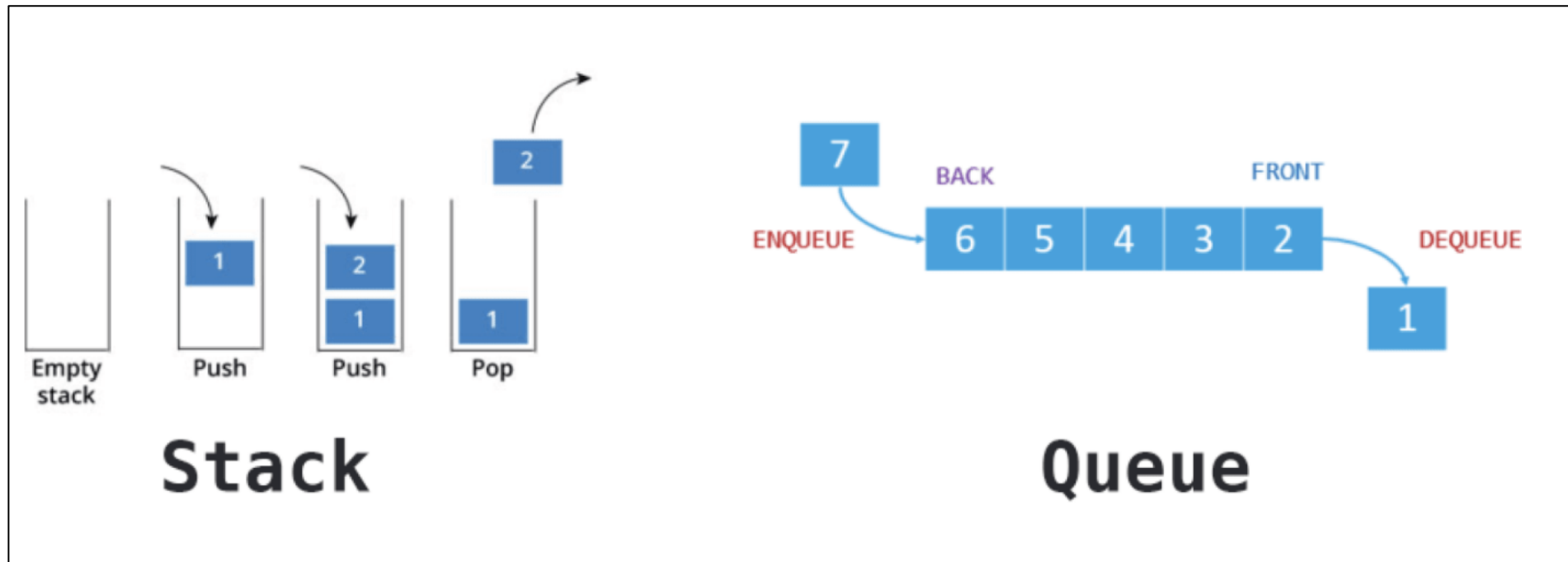
- [8680 열린 괄호](#)
- [1221 후위표기법1](#)
- [2858 쇠막대기](#)

2. Queue

2. Queue

Queue(큐)

- 데이터를 한쪽 끝(뒤)에서 넣고 반대쪽 끝(앞)에서 뺄 수 있는 선입선출(FIFO, First-In First-Out) 방식의 자료구조



2. Queue

```

1  #include <iostream>
2  #include <queue>
3
4  using namespace std;
5
6  int main() {
7      queue<int> q({1, 2, 3, 4, 5, 6});
8
9      q.push(7);
10     q.pop();
11
12     return 0;
13 }

```

queue<자료형> 변수명 | <queue>

- 큐를 선언하기 위해서는 <queue> 라이브러리 필요
- 자료형에는 거의 모든 자료형을 사용할 수도 있으며, 구조체를 선언하여 자료형으로 넣을 수도 있음
- queue<자료형> 변수명({요소1, 요소2, 요소3}) 처럼 초기 선언도 가능하며, 이 경우 요소1부터 순서대로 push함.

q.push(요소) | <queue> | O(1)

- 큐의 뒷부분(Rear)에 새로운 요소를 추가하여 저장하는 작업
- 이론적으로는 enqueue라고 하지만, C++에서는 통일성을 위해 push로 통일 됨

q.pop() | <queue> | O(1)

- 큐의 가장 앞부분(Front)에 위치한 요소를 제거하는 작업
- 비어 있는 큐에 pop을 사용하면 프로그램이 정지되며 아무것도 출력되지 않음
- 이론적으로는 dequeue라고 하지만, C++에서는 통일성을 위해 pop으로 통일 됨



2. Queue

```
1  #include <iostream>
2  #include <queue>
3
4  using namespace std;
5
6  int main() {
7      queue<int> q({5, 8, 4, 3, 2});
8
9      cout << q.front() << "\n";
10     cout << q.back() << "\n";
11     cout << q.size() << "\n";
12
13     q.push(q.front());
14
15     cout << q.back() << "\n";
16     cout << q.empty();
17 }
```

Input: Copy

Expected Output: Copy

```
5
2
5
5
0
```

q.front() | <queue> | O(1)

- 큐의 가장 앞에 위치한 데이터를 조회하는 함수
- 비어 있는 큐에 front를 사용하면 프로그램이 정지되며 아무것도 출력되지 않음

q.back() | <queue> | O(1)

- 큐의 가장 뒷부분에 위치한 데이터를 조회하는 함수
- 비어 있는 큐에 back을 사용하면 프로그램이 정지되며 아무것도 출력되지 않음

q.size() | <queue> | O(1)

- 큐에 현재 저장되어 있는 데이터의 총 개수를 확인하는 함수
- 내부 컨테이너에 요소의 수를 저장하고 있어서 O(1)의 시간 복잡도를 가짐

q.empty() | <queue> | O(1)

- 큐에 저장된 데이터가 없는지 여부를 확인하는 함수
- bool형으로 반환하며, 큐가 비어 있으면 true, 요소가 하나도 존재한다면 false를 반환

큐의 초기화 | O(n)

- 큐에는 q.clear() 함수가 존재하지 않음
- 큐를 초기화하기 위해서는 재선언 하거나 q = {}으로 초기화할 수 있음
- 단, q = {1, 2, 3} 같은 방법은 통하지 않고, q = {}만 예외적으로 가능함.

2. Queue

- 18258 큐 2

- 코드

```
1  #include <iostream>
2  #include <queue>
3  #include <string>
4
5  using namespace std;
6
7  int main() {
8      ios_base::sync_with_stdio(false);
9      cin.tie(NULL);
10
11     int n, num;
12     string s;
13     queue<int> q;
14
15     cin >> n;
```

```
17     for(int i = 0; i < n; i++) {
18         cin >> s;
19
20         if(s == "push") {
21             cin >> num;
22
23             q.push(num);
24         }
25         else if(s == "size") cout << q.size() << "\n";
26         else if(s == "empty") {
27             if(q.empty()) cout << "1\n";
28             else cout << "0\n";
29         }
30         else if(!q.empty()) {
31             if(s == "pop") {
32                 cout << q.front() << "\n";
33
34                 q.pop();
35             }
36             else if(s == "front") cout << q.front() << "\n";
37             else cout << q.back() << "\n";
38         }
39         else cout << "-1\n";
40     }
41 }
```

2. Queue: 실습(BOJ)



실습

- [2164 카드2](#)
 - [코드](#)
- [15828 Router](#)
 - [코드](#)

2. Queue: 실습(정올)



실습

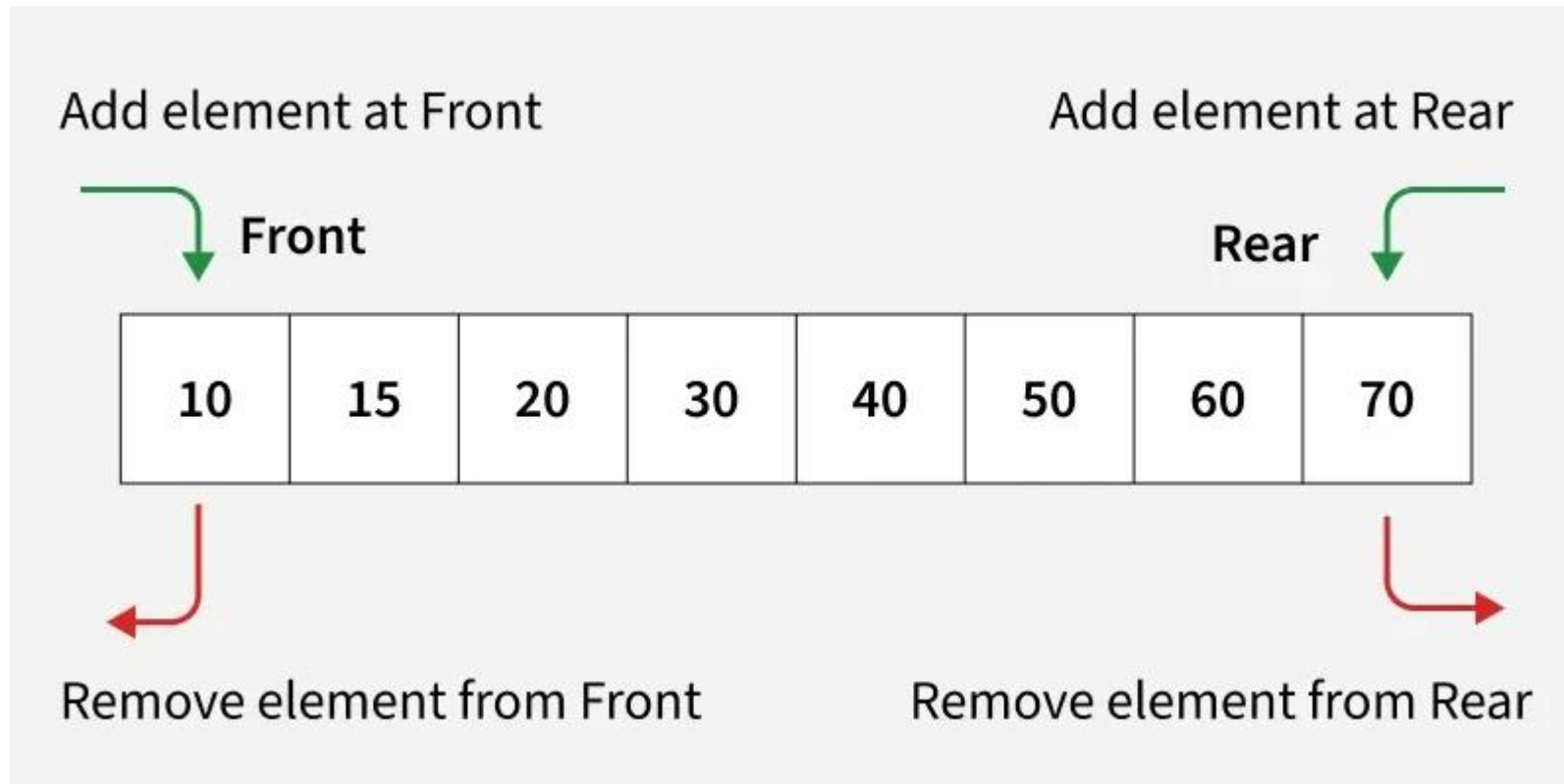
- [1697 큐\(queue\)](#)
- [6361 피자 데이\(Pizza Day\)](#)
- [5527 콜센터](#)

3. Deque

3. Deque

Deque(덱/데크)

- 양쪽 끝에서 데이터를 넣고 뺄 수 있는 양방향 자료구조 (Double-Ended Queue)



3. Deque

deque<자료형> 변수명 | <deque>

- 덱을 선언하기 위해서는 <deque> 라이브러리 필요
- 자료형에는 거의 모든 자료형을 사용할 수도 있으며, 구조체를 선언하여 자료형으로 넣을 수도 있음

deq.push_front(요소), deq.push_back(요소) | <deque> | $O(1)$

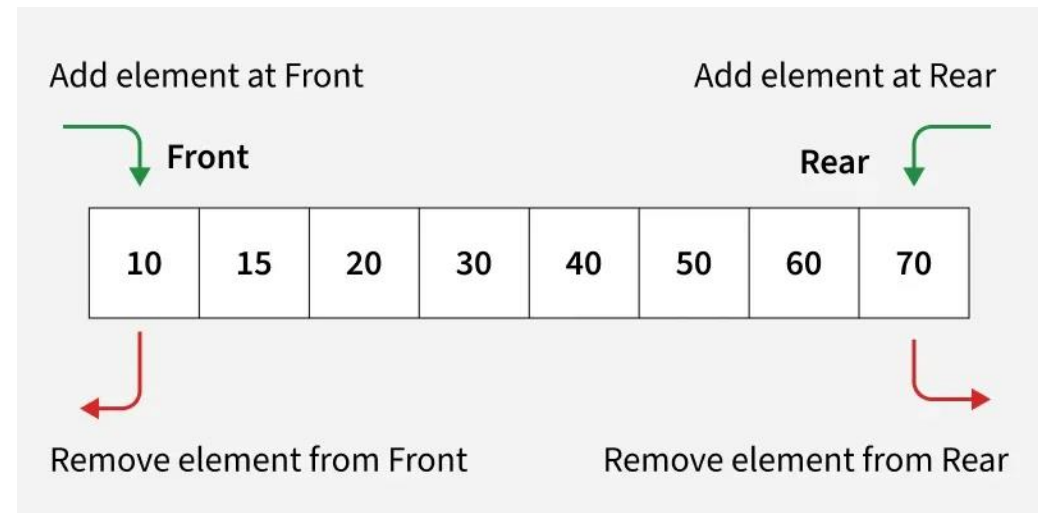
- 덱의 가장 앞, 뒷부분에 새로운 데이터를 추가하여 저장하는 함수

deq.pop_front(), deq.pop_back() | <deque> | $O(1)$

- 덱의 가장 앞, 뒷부분에 위치한 데이터를 제거하는 작업
- 비어 있는 덱에 pop을 사용하면 프로그램이 정지되며 아무것도 출력되지 않음

```

1  #include <iostream>
2  #include <deque>
3
4  using namespace std;
5
6  int main() {
7      deque<int> deq({15, 20, 30, 40, 50, 60});
8
9      deq.push_front(10);
10     deq.push_back(70);
11
12     deq.pop_front();
13     deq.pop_back();
14
15     return 0;
16 }
```



3. Deque

```
1  #include <iostream>
2  #include <deque>
3
4  using namespace std;
5
6  int main() {
7      deque<char> deq({'h', 'e', 'l', 'l', 'o'});
8
9      cout << deq.front() << "\n";
10     cout << deq.back() << "\n";
11     cout << deq.size() << "\n";
12     cout << deq.empty();
13
14     return 0;
15 }
```

Input:

Copy

Expected Output:

Copy

h
o
5
0

deq.front(), deq.back() | `<deque>` | $O(1)$

- 덱의 가장 윗부분에 위치한 데이터를 조회하는 함수
- 비어 있는 덱에 조회 함수를 사용하면 프로그램이 정지되며 아무것도 출력되지 않음

deq.size() | `<deque>` | $O(1)$

- 덱에 현재 저장되어 있는 데이터의 총 개수를 확인하는 함수
- 내부 컨테이너에 요소의 수를 저장하고 있어서 $O(1)$ 의 시간 복잡도를 가짐

deq.empty() | `<deque>` | $O(1)$

- 덱에 저장된 데이터가 없는지 여부를 확인하는 함수
- bool형으로 반환하며, stack이 비어 있으면 true, 요소가 하나도 존재한다면 false를 반환

3. Deque: 차이점을 중심으로

```
1  #include <iostream>
2  #include <deque>
3
4  using namespace std;
5
6  int main() {
7      deque<int> deq = {3, 2, 1};
8
9      cout << deq.front() << "\n";
10
11     deq = {4, 5, 6};
12
13     cout << deq.front() << "\n";
14     cout << deq[1] << "\n";
15
16     deq.clear();
17
18     cout << deq.empty();
19
20     return 0;
21 }
```

deq.clear() | <deque> | $O(N)$

- deq.clear()를 통해 덱의 내부 요소를 제거하는 초기화 작업을 할 수 있음

덱의 초기값 | <deque> | $O(1)$

- 스택과 큐에서는 생성자를 통해서만 초기값을 지정할 수 있었던 것과 다르게 자유롭게 초기값을 재 지정할 수 있음

덱의 인덱싱 | <deque> | $O(1)$

- 스택과 큐는 인덱싱이 불가능 했던 것과 다르게 벡터처럼 인덱싱이 가능함.

Input:

Copy

Expected Output:

Copy

3
4
5
1

3. Deque: 사실은...

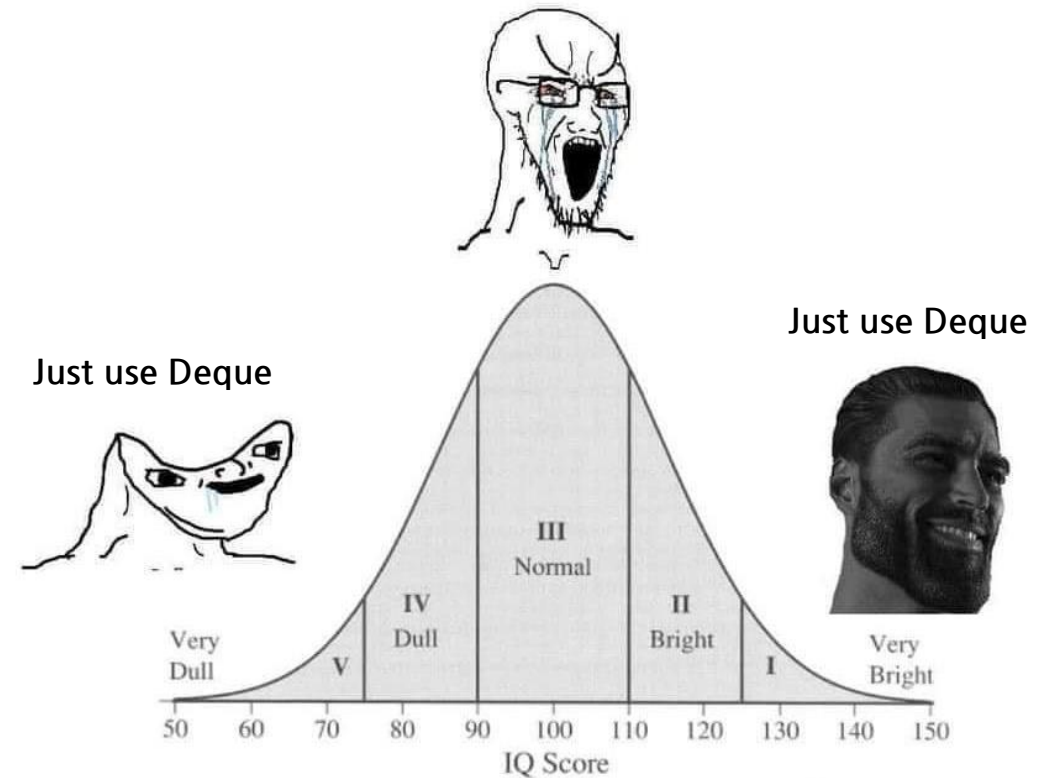
C++에서 스택과 큐는 내부적으로 덱을 사용함

- 덱에서 각 자료구조에 맞게 기능을 제한한 형태임.
- 시간복잡도, 공간복잡도 관점에서 차이가 전혀 없음.

이 문제는 후입선출을 요구하는 문제니 스택을 써야하고,
저 문제는 선입선출을 요구하니 큐를 쓰고...

그럼에도 스택과 큐를 사용하는 이유

- 꼭 필요한 기능만 남겨두어 실수를 차단하는 것에 목적이 있음



3. Deque

- 28279 덱 2
- 코드

```
1  #include <iostream>
2  #include <deque>
3
4  using namespace std;
5
6  int main() {
7      ios::sync_with_stdio(false);
8      cin.tie(NULL);
9
10     int n, com, num;
11     deque<int> deq;
12
13     cin >> n;
```

```
15     for(int i = 0; i < n; i++) {
16         cin >> com;
17
18         if(com == 1) {
19             cin >> num;
20
21             deq.push_front(num);
22         }
23         else if(com == 2) {
24             cin >> num;
25
26             deq.push_back(num);
27         }
28         else if(com == 5) cout << deq.size() << "\n";
29         else if(com == 6) {
30             if(deq.empty()) cout << "1 \n";
31             else cout << "0 \n";
32         }
```

```
33     else {
34         if(deq.empty()) cout << "-1 \n";
35         else {
36             if(com == 3) {
37                 cout << deq.front() << "\n";
38
39                 deq.pop_front();
40             }
41             else if(com == 4) {
42                 cout << deq.back() << "\n";
43
44                 deq.pop_back();
45             }
46             else if(com == 7) cout << deq.front() << "\n";
47             else cout << deq.back() << "\n";
48         }
49     }
50 }
```

3. Deque: 실습(BOJ)

실습

- [24511 queuestack](#)
- [코드](#)

```
4
0 1 1 0
1 2 3 4
```

각 자료구조(0 : queue, 1 : stack)에 값을 하나씩 넣어두는 과정이라고 생각하면 된다.
이렇게 개별적인 큐와 스택을 모아둔 전체를 이 문제에서는 queuestack이라고 한다.

